

Lecture 9:

Neural networks

Vegard Høghaug Larsen

GRA4160: Predictive modelling with machine learning

March 4th 2026

Plan for today:

- ▶ Introduction to neural networks
- ▶ Single layer (feed forward) neural networks
- ▶ Fitting a Neural Network

- ▶ A neural network is a computational system that is inspired by the structure and function of the human brain

- ▶ A neural network is a computational system that is inspired by the structure and function of the human brain
- ▶ It consists of many interconnected processing units, called neurons, that work together to solve complex problems

- ▶ A neural network is a computational system that is inspired by the structure and function of the human brain
- ▶ It consists of many interconnected processing units, called neurons, that work together to solve complex problems
- ▶ Each neuron receives input from other neurons, processes that input, and then produces an output that is sent to other neurons

- ▶ A neural network is a computational system that is inspired by the structure and function of the human brain
- ▶ It consists of many interconnected processing units, called neurons, that work together to solve complex problems
- ▶ Each neuron receives input from other neurons, processes that input, and then produces an output that is sent to other neurons
- ▶ By combining and processing information in this way, a neural network can learn to recognize patterns and make predictions

Single layer (feed forward) neural networks

- ▶ A single layer neural network is a model that takes a set of p input variables $X = (X_1, X_2, \dots, X_p)$

Single layer (feed forward) neural networks

- ▶ A single layer neural network is a model that takes a set of p input variables $X = (X_1, X_2, \dots, X_p)$
- ▶ The NN is a nonlinear function $f(X)$ that maps the input variables to an output variable Y

Single layer (feed forward) neural networks

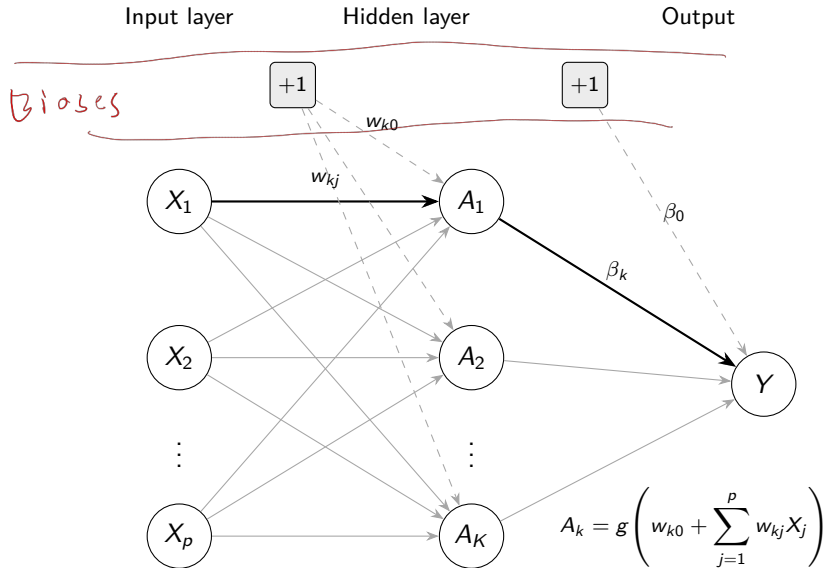
- ▶ A single layer neural network is a model that takes a set of p input variables $X = (X_1, X_2, \dots, X_p)$
- ▶ The NN is a nonlinear function $f(X)$ that maps the input variables to an output variable Y
- ▶ The weights $w_1, w_2, \dots, w_p$ are the parameters of the model, controlling the strength of the relationship between the input variables and the output variable

Single layer (feed forward) neural networks

- ▶ A single layer neural network is a model that takes a set of p input variables $X = (X_1, X_2, \dots, X_p)$
- ▶ The NN is a nonlinear function $f(X)$ that maps the input variables to an output variable Y
- ▶ The weights $w_1, w_2, \dots, w_p$ are the parameters of the model, controlling the strength of the relationship between the input variables and the output variable
- ▶ Given a nonlinear activation function $g(z)$, the model can be written as a sum of K activation functions

$$Y = f(X) = \beta_0 + \sum_{k=1}^K \beta_k g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_j \right)$$

Neural network with a single hidden layer



Hidden units and the output layer

The network is built up in two steps:

1. Specify the number of hidden units K and the activation function $g(X)$ and we can calculate K activations in the hidden layer as

$$A_k = h_k(X) = g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_j \right)$$

Hidden units and the output layer

The network is built up in two steps:

1. Specify the number of hidden units K and the activation function $g(X)$ and we can calculate K activations in the hidden layer as

$$A_k = h_k(X) = g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_j \right)$$

2. The K activations feed into the output layer, which is a linear function of the activations

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k A_k$$

The forward pass as matrix multiplications

We can write the forward pass compactly using matrices. Let $\mathbf{X}$ be the $n \times p$ input matrix and define:

The forward pass as matrix multiplications

We can write the forward pass compactly using matrices. Let $\mathbf{X}$ be the $n \times p$ input matrix and define:

1. **Hidden layer:** Compute the $n \times K$ matrix of activations

$$A_{ik} = g\left(w_{k0} + \sum_{j=1}^p w_{kj} X_{ij}\right) \iff \mathbf{A} = g\left(\mathbf{1}_n \mathbf{w}_0^\top + \mathbf{X} \mathbf{W}^\top\right)$$

where $\mathbf{W}$ is the $K \times p$ weight matrix, $\mathbf{w}_0$ is the $K \times 1$ bias vector, and $g(\cdot)$ is applied element-wise

The forward pass as matrix multiplications

We can write the forward pass compactly using matrices. Let $\mathbf{X}$ be the $n \times p$ input matrix and define:

1. **Hidden layer:** Compute the $n \times K$ matrix of activations

$$A_{ik} = g\left(w_{k0} + \sum_{j=1}^p w_{kj} X_{ij}\right) \iff \mathbf{A} = g(\mathbf{1}_n \mathbf{w}_0^\top + \mathbf{X} \mathbf{W}^\top)$$

where $\mathbf{W}$ is the $K \times p$ weight matrix, $\mathbf{w}_0$ is the $K \times 1$ bias vector, and $g(\cdot)$ is applied element-wise

2. **Output layer:** Compute the $n \times 1$ output vector

$$Y_i = \beta_0 + \sum_{k=1}^K \beta_k A_{ik} \iff \mathbf{Y} = \beta_0 \mathbf{1}_n + \mathbf{A} \boldsymbol{\beta}$$

where $\boldsymbol{\beta} = (\beta_1, \dots, \beta_K)^\top$

The forward pass as matrix multiplications

We can write the forward pass compactly using matrices. Let $\mathbf{X}$ be the $n \times p$ input matrix and define:

1. **Hidden layer:** Compute the $n \times K$ matrix of activations

$$A_{ik} = g\left(w_{k0} + \sum_{j=1}^p w_{kj} X_{ij}\right) \iff \mathbf{A} = g(\mathbf{1}_n \mathbf{w}_0^\top + \mathbf{XW}^\top)$$

where $\mathbf{W}$ is the $K \times p$ weight matrix, $\mathbf{w}_0$ is the $K \times 1$ bias vector, and $g(\cdot)$ is applied element-wise

2. **Output layer:** Compute the $n \times 1$ output vector

$$Y_i = \beta_0 + \sum_{k=1}^K \beta_k A_{ik} \iff \mathbf{Y} = \beta_0 \mathbf{1}_n + \mathbf{A}\boldsymbol{\beta}$$

where $\boldsymbol{\beta} = (\beta_1, \dots, \beta_K)^\top$

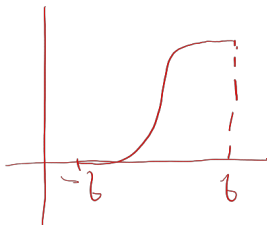
Each layer is a matrix multiplication followed by an activation function — this is what makes neural networks efficient to compute on GPUs

What does the activation function look like?

Activation functions are used to introduce non-linearity and enable the network to learn complex patterns in the input data. Here are some common activation functions used in neural networks:

1. The sigmoid function (we also used this for logistic regression)

$$g(z) = \frac{1}{1 + e^{-z}}$$



both up to 100
issues with 101
still gives close to 1

What does the activation function look like?

Activation functions are used to introduce non-linearity and enable the network to learn complex patterns in the input data. Here are some common activation functions used in neural networks:

1. The sigmoid function (we also used this for logistic regression)

$$g(z) = \frac{1}{1 + e^{-z}}$$

2. Rectified Linear Unit (ReLU)

$$g(z) = \max(0, z)$$

bank to squashing
whenever info

What does the activation function look like?

Activation functions are used to introduce non-linearity and enable the network to learn complex patterns in the input data. Here are some common activation functions used in neural networks:

1. The sigmoid function (we also used this for logistic regression)

$$g(z) = \frac{1}{1 + e^{-z}}$$

2. Rectified Linear Unit (ReLU)

$$g(z) = \max(0, z)$$

GELN used
in the search

3. Softmax (typically used in the output layer for multi-class classification)

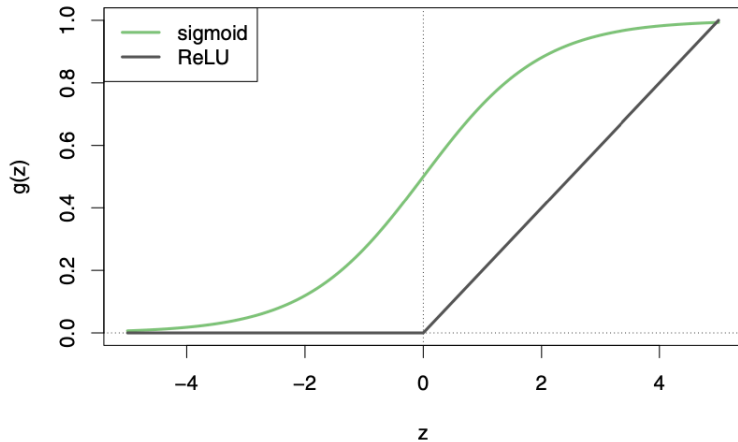
$$g(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}$$

gives prob. dist as output
Denny

tanh not known
sigmoid

temperature in LLMs ≈ 0 Autoconfig

Sigmoid vs ReLU



Source: An Introduction to Statistical Learning, James, Witten, Hastie and Tibshirani, 2021

We can fit the parameters of a neural network by minimizing a cost function (nonlinear least squares)

$$\min_{w_k, \beta} \frac{1}{2} \sum_{i=1}^n (y_i - f(x_i))^2$$

where

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_j \right)$$

We can fit the parameters of a neural network by minimizing a cost function (nonlinear least squares)

$$\min_{w_k, \beta} \frac{1}{2} \sum_{i=1}^n (y_i - f(x_i))^2$$

where

$$f(X) = \beta_0 + \sum_{k=1}^K \beta_k g \left(w_{k0} + \sum_{j=1}^p w_{kj} X_j \right)$$

This is a nonlinear optimization problem, and it is not straightforward to solve.

Gradient descent

We can use gradient descent to find the optimal parameters. Let's put all the parameters in one vector θ and define the cost function as

$$L(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2$$

We can use gradient descent to find the optimal parameters. Let's put all the parameters in one vector θ and define the cost function as

$$L(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2$$

Then, to find the optimal parameters, we can use gradient descent to minimize the cost function $L(\theta)$ by the following iterative update rule:

1. Let θ^t be a vector of all the parameters in the system at iteration t . Start with an initial guess θ^0

Gradient descent

We can use gradient descent to find the optimal parameters. Let's put all the parameters in one vector θ and define the cost function as

$$L(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2$$

Then, to find the optimal parameters, we can use gradient descent to minimize the cost function $L(\theta)$ by the following iterative update rule:

1. Let θ^t be a vector of all the parameters in the system at iteration t . Start with an initial guess θ^0
2. Update the parameters according to the following rule: $\theta^{t+1} = \theta^t + \delta$, where δ is a vector that reflects a small change in the parameters such that

$$L(\theta^{t+1}) < L(\theta^t)$$

3. Repeat until convergence

How do we know how to update the parameters (what is δ)?

We need to calculate the gradient of the cost function $L(\theta)$ with respect to the parameters θ :

$$\nabla L(\theta^t) = \left. \frac{\partial L(\theta)}{\partial \theta} \right|_{\theta=\theta^t} = \left. \frac{\partial}{\partial \theta} \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 \right|_{\theta=\theta^t}$$

How do we know how to update the parameters (what is δ)?

We need to calculate the gradient of the cost function $L(\theta)$ with respect to the parameters θ :

$$\nabla L(\theta^t) = \left. \frac{\partial L(\theta)}{\partial \theta} \right|_{\theta=\theta^t} = \left. \frac{\partial}{\partial \theta} \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 \right|_{\theta=\theta^t}$$

$\nabla L(\theta^t)$ gives us the directions to move θ so as to decrease the objective $L(\theta)$

How do we know how to update the parameters (what is δ)?

We need to calculate the gradient of the cost function $L(\theta)$ with respect to the parameters θ :

$$\nabla L(\theta^t) = \left. \frac{\partial L(\theta)}{\partial \theta} \right|_{\theta=\theta^t} = \left. \frac{\partial}{\partial \theta} \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 \right|_{\theta=\theta^t}$$

$\nabla L(\theta^t)$ gives us the directions to move θ so as to decrease the objective $L(\theta)$

Given a learning rate η , we can use the gradient to update the parameters as follows:

$$\theta^{t+1} = \theta^t - \eta \nabla L(\theta^t)$$

How do we know how to update the parameters (what is δ)?

We need to calculate the gradient of the cost function $L(\theta)$ with respect to the parameters θ :

$$\nabla L(\theta^t) = \left. \frac{\partial L(\theta)}{\partial \theta} \right|_{\theta=\theta^t} = \left. \frac{\partial}{\partial \theta} \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 \right|_{\theta=\theta^t}$$

$\nabla L(\theta^t)$ gives us the directions to move θ so as to decrease the objective $L(\theta)$

Given a learning rate η , we can use the gradient to update the parameters as follows:

$$\theta^{t+1} = \theta^t - \eta \nabla L(\theta^t)$$

The gradient packs all the partial derivatives into a single vector: $\nabla L = \left[\frac{\partial L}{\partial \beta_k}, \frac{\partial L}{\partial w_{kj}} \right]^T$

Backpropagation: setup

Recall the forward pass for observation i . Define the intermediate quantities:

$$z_{ik} = w_{k0} + \sum_{j=1}^p w_{kj} x_{ij} \quad (\text{pre-activation for hidden unit } k)$$

$$A_{ik} = g(z_{ik}) \quad (\text{activation for hidden unit } k)$$

$$f_i = \beta_0 + \sum_{k=1}^K \beta_k A_{ik} \quad (\text{network output})$$

$$L_i = \frac{1}{2} (y_i - f_i)^2 \quad (\text{loss for observation } i)$$

Backpropagation: setup

Recall the forward pass for observation i . Define the intermediate quantities:

$$z_{ik} = w_{k0} + \sum_{j=1}^p w_{kj} x_{ij} \quad (\text{pre-activation for hidden unit } k)$$

$$A_{ik} = g(z_{ik}) \quad (\text{activation for hidden unit } k)$$

$$f_i = \beta_0 + \sum_{k=1}^K \beta_k A_{ik} \quad (\text{network output})$$

$$L_i = \frac{1}{2} (y_i - f_i)^2 \quad (\text{loss for observation } i)$$

The total loss is $L(\theta) = \sum_{i=1}^n L_i$. We need $\frac{\partial L_i}{\partial \beta_k}$ and $\frac{\partial L_i}{\partial w_{kj}}$.

The key idea: apply the **chain rule** layer by layer, working *backwards* from the loss to the parameters.

Backpropagation: the chain rule

Step 1: The common starting point — derivative of the loss w.r.t. the output:

$$\frac{\partial L_i}{\partial f_i} = -(y_i - f_i)$$

Backpropagation: the chain rule

Step 1: The common starting point — derivative of the loss w.r.t. the output:

$$\frac{\partial L_i}{\partial f_i} = -(y_i - f_i)$$

Step 2: Derivative w.r.t. the output-layer weights β_k (one link in the chain):

$$\frac{\partial L_i}{\partial \beta_k} = \underbrace{\frac{\partial L_i}{\partial f_i}}_{-(y_i - f_i)} \cdot \underbrace{\frac{\partial f_i}{\partial \beta_k}}_{A_{ik}} = -(y_i - f_i) \cdot A_{ik}$$

Backpropagation: the chain rule

Step 1: The common starting point — derivative of the loss w.r.t. the output:

$$\frac{\partial L_i}{\partial f_i} = -(y_i - f_i)$$

Step 2: Derivative w.r.t. the output-layer weights β_k (one link in the chain):

$$\frac{\partial L_i}{\partial \beta_k} = \underbrace{\frac{\partial L_i}{\partial f_i}}_{-(y_i - f_i)} \cdot \underbrace{\frac{\partial f_i}{\partial \beta_k}}_{A_{ik}} = -(y_i - f_i) \cdot A_{ik}$$

Step 3: Derivative w.r.t. the hidden-layer weights w_{kj} (three links in the chain):

$$\frac{\partial L_i}{\partial w_{kj}} = \underbrace{\frac{\partial L_i}{\partial f_i}}_{-(y_i - f_i)} \cdot \underbrace{\frac{\partial f_i}{\partial A_{ik}}}_{\beta_k} \cdot \underbrace{\frac{\partial A_{ik}}{\partial z_{ik}}}_{g'(z_{ik})} \cdot \underbrace{\frac{\partial z_{ik}}{\partial w_{kj}}}_{x_{ij}} = -(y_i - f_i) \cdot \beta_k \cdot g'(z_{ik}) \cdot x_{ij}$$

Backpropagation: the chain rule

Step 1: The common starting point — derivative of the loss w.r.t. the output:

$$\frac{\partial L_i}{\partial f_i} = -(y_i - f_i)$$

Step 2: Derivative w.r.t. the output-layer weights β_k (one link in the chain):

$$\frac{\partial L_i}{\partial \beta_k} = \underbrace{\frac{\partial L_i}{\partial f_i}}_{-(y_i - f_i)} \cdot \underbrace{\frac{\partial f_i}{\partial \beta_k}}_{A_{ik}} = -(y_i - f_i) \cdot A_{ik}$$

Step 3: Derivative w.r.t. the hidden-layer weights w_{kj} (three links in the chain):

$$\frac{\partial L_i}{\partial w_{kj}} = \underbrace{\frac{\partial L_i}{\partial f_i}}_{-(y_i - f_i)} \cdot \underbrace{\frac{\partial f_i}{\partial A_{ik}}}_{\beta_k} \cdot \underbrace{\frac{\partial A_{ik}}{\partial z_{ik}}}_{g'(z_{ik})} \cdot \underbrace{\frac{\partial z_{ik}}{\partial w_{kj}}}_{x_{ij}} = -(y_i - f_i) \cdot \beta_k \cdot g'(z_{ik}) \cdot x_{ij}$$

Each parameter receives a fraction of the residual $(y_i - f_i)$ propagated *back* through the network — hence the name **backpropagation**.

Stochastic gradient descent (SGD)

- ▶ SGD is a method for minimizing a cost function by iteratively moving in the direction of steepest descent as defined by the negative of the gradient

Stochastic gradient descent (SGD)

- ▶ SGD is a method for minimizing a cost function by iteratively moving in the direction of steepest descent as defined by the negative of the gradient
- ▶ In contrast to batch gradient descent, which computes the gradient of the cost function over the entire dataset, SGD computes the gradient on a single training example at a time

Stochastic gradient descent (SGD)

- ▶ SGD is a method for minimizing a cost function by iteratively moving in the direction of steepest descent as defined by the negative of the gradient
- ▶ In contrast to batch gradient descent, which computes the gradient of the cost function over the entire dataset, SGD computes the gradient on a single training example at a time
- ▶ This makes SGD much faster than batch gradient descent, but it also means that SGD is much noisier

Stochastic gradient descent (SGD)

- ▶ SGD is a method for minimizing a cost function by iteratively moving in the direction of steepest descent as defined by the negative of the gradient
- ▶ In contrast to batch gradient descent, which computes the gradient of the cost function over the entire dataset, SGD computes the gradient on a single training example at a time
- ▶ This makes SGD much faster than batch gradient descent, but it also means that SGD is much noisier
- ▶ The noise can be reduced by averaging the gradients over several training examples

- ▶ L2 regularization adds a penalty term to the cost function to prevent overfitting
- ▶ The penalty term is proportional to the sum of the squares of the weights in the network
- ▶ The regularized cost function is

$$L_{\text{reg}}(\theta) = \frac{1}{2} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2 + \lambda \sum_{k,j} w_{kj}^2$$

where λ is the regularization parameter controlling the trade-off between fit and complexity

Dropout

think LLM, Overfitting

- ▶ Dropout is a regularization technique that randomly drops out (sets to zero) some of the neurons in the network during training
- ▶ This helps prevent the network from overfitting to the training data
- ▶ Dropout can be seen as an ensemble of all the networks that can be formed by dropping out different neurons
- ▶ Typical dropout rates are between 0.2 and 0.5, and dropout is only applied during training, not at inference time

help Break up